Hill Cimbing

```
def cost(state):
    conflicts = 0
    n = len(state)

    for i in range(n):
        for j in range(i + 1, n):
            if (state[i] == state[j] or
                abs(state[i] - state[j]) == abs(i - j)):
                conflicts += 1

    return conflicts


def print_board(state):
    n = len(state)

    for row in range(n - 1, -1, -1):
        for col in range(n):
            if state[col] == row:
                print(" Q ", end="")
            else:
                print(" . ", end="")
        print()
    print()


def hill_climbing():
    state = [3, 1, 2, 0]
    n = len(state)
    step = 1

    print("INITIAL BOARD:")
    print_board(state)
    print("Initial State:", state)
    print("Initial Cost:", cost(state))

    while cost(state) != 0:
        current_cost = cost(state)
        neighbors = []
```

```
    # Generate neighbors by swapping row positions
    for i in range(n):
        for j in range(i + 1, n):
            neighbor = state.copy()
            neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
            neighbors.append((cost(neighbor), i, j, neighbor))

    # Select minimum-cost neighbor; break ties by indices
    neighbors.sort(key=lambda x: (x[0], x[1], x[2]))

    best_cost, i, j, best_state = neighbors[0]

    print("\nIteration:", step)
    print("Current State:", state)
    print("Current Cost:", current_cost)

    for neighbor_cost, r1, r2, neighbor in neighbors:
        print("Swap rows:", r1, "and", r2,
              "| State:", neighbor,
              "| Cost:", neighbor_cost)

    print("Selected Swap:", i, "and", j)
    print("Selected State:", best_state)
    print("Selected Cost:", best_cost)

    if best_cost >= current_cost:
        print("\nNo better neighbor found. Stopping.")
        print("Final Cost:", current_cost)
        print("Final State:", state)
        print("FINAL BOARD:")
        print_board(state)
        return

    state = best_state
    step += 1

print("\nGOAL STATE REACHED!")
print("Final State:", state)
print("Final Cost:", cost(state))
```

```
    print("FINAL BOARD:")
    print_board(state)


hill_climbing()
```

```
INITIAL BOARD:
Q . . .
. . Q .
. Q . .
. . . Q

Initial State: [3, 1, 2, 0]
Initial Cost: 2

Iteration: 1
Current State: [3, 1, 2, 0]
Current Cost: 2
Swap rows: 0 and 1 | State: [1, 3, 2, 0] | Cost: 1
Swap rows: 0 and 2 | State: [2, 1, 3, 0] | Cost: 1
Swap rows: 1 and 3 | State: [3, 0, 2, 1] | Cost: 1
Swap rows: 2 and 3 | State: [3, 1, 0, 2] | Cost: 1
Swap rows: 0 and 3 | State: [0, 1, 2, 3] | Cost: 6
Swap rows: 1 and 2 | State: [3, 2, 1, 0] | Cost: 6
Selected Swap: 0 and 1
Selected State: [1, 3, 2, 0]
Selected Cost: 1

Iteration: 2
Current State: [1, 3, 2, 0]
Current Cost: 1
Swap rows: 2 and 3 | State: [1, 3, 0, 2] | Cost: 0
Swap rows: 0 and 1 | State: [3, 1, 2, 0] | Cost: 2
Swap rows: 0 and 2 | State: [2, 3, 1, 0] | Cost: 2
Swap rows: 1 and 3 | State: [1, 0, 2, 3] | Cost: 2
Swap rows: 0 and 3 | State: [0, 3, 2, 1] | Cost: 4
Swap rows: 1 and 2 | State: [1, 2, 3, 0] | Cost: 4
Selected Swap: 2 and 3
Selected State: [1, 3, 0, 2]
Selected Cost: 0

GOAL STATE REACHED!
Final State: [1, 3, 0, 2]
```

Simulated Annealing:

```
import random
import math

def cost(state):
    conflicts = 0
    for i in range(8):
        for j in range(i + 1, 8):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i -
j):
                conflicts += 1
    return conflicts


def print_grid(state):
    print("  ", end="")
    for col in range(1, 9):
```

```
        print(col, end=" ")
    print()

    for row in range(1, 9):
        print(row, end=" ")
        for col in range(8):
            if state[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
    print()


def simulated_annealing():
    current = [random.randint(1, 8) for _ in range(8)]
    initial = current.copy()

    T = 1000
    cooling_rate = 0.99

    print("INITIAL STATE:", initial)
    print("Initial Cost:", cost(initial))
    print("Initial Board:")
    print_grid(initial)

    while T > 0.1:
        if cost(current) == 0:
            break

        next_state = current.copy()
        col = random.randint(0, 7)
        row = random.randint(1, 8)
        next_state[col] = row

        deltaE = cost(current) - cost(next_state)

        if deltaE > 0:
            current = next_state
        elif random.random() < math.exp(deltaE / T):
```

```
            current = next_state

        T *= cooling_rate

    print("FINAL STATE:", current)
    print("Final Cost:", cost(current))
    print("Final Board:")
    print_grid(current)

    if cost(current) == 0:
        print("Solution found!")
    else:
        print("No solution found. Run again.")


solution = simulated_annealing()
```

Output:

```
INITIAL STATE: [7, 3, 6, 5, 6, 8, 1, 4]
Initial Cost: 6
Initial Board:
  1 2 3 4 5 6 7 8
1 . . . . . Q . .
2 . . . . . . . .
3 . Q . . . . . .
4 . . . . . . . Q
5 . . . Q . . . .
6 . . Q . Q . . .
7 Q . . . . . . .
8 . . . . . Q . .

FINAL STATE: [3, 6, 2, 7, 5, 1, 8, 4]
Final Cost: 0
Final Board:
  1 2 3 4 5 6 7 8
1 . . . . . Q . .
2 . . Q . . . . .
3 Q . . . . . . .
4 . . . . . . . Q
5 . . . . Q . . .
6 . Q . . . . . .
7 . . . Q . . . .
8 . . . . . Q . .

Solution found!
```